Lecture 25

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Review of Lecture 24: Linear Solve as Minimization

- Quadratic form:

$$f(x) = \frac{1}{2}x^T Ax - b^T x + c, \qquad \nabla f = \frac{1}{2}(A^T + A)x - b$$

- If A is symmetric, $\nabla f = Ax - b = -r$, so $Ax = b$ at the critical point of f.
- If A is also positive definite, f is concave up and that point is the unique minimum.
- Line search: $x_{k+1} = x_k + \alpha s_k$, with α chosen so that $r_{k+1}^T s_k = 0$:

$$\alpha = \frac{r_k^T s_k}{s_k^T A s_k}$$

- Steepest descent: $s_k = r_k$, so $\alpha = \dfrac{r_k^T r_k}{r_k^T A r_k}$.
- Residual update without forming $b - Ax$: $r_{k+1} = r_k - \alpha A s_k$.

Review of Lecture 24: Conjugate Directions

- Steepest descent zig-zags; better to choose search directions that are A-*conjugate*: $u^T A v = 0$.
- Start with $s_0 = r_0 = b - Ax_0$.
- Next direction: $s_{k+1} = r_{k+1} + \beta_{k+1} s_k$, with β chosen so that $s_{k+1}^T A s_k = 0$:

$$\beta = -\frac{r_{k+1}^T A s_k}{s_k^T A s_k}$$

- After simplification: only one matrix-vector product As_k per step.
- Result: CG converges in at most n steps (in exact arithmetic).

Today: the CG algorithm, convergence, preconditioning — then Markov chains and Google.

Conjugate Gradients

$x_0 =$ `initial guess`
$r_0 = b - Ax_0$
$s_0 = r_0$
for $k = 0, 1, 2, \ldots$
    $\alpha_k = \frac{r_k^T r_k}{s_k^T A s_k}$
    $x_{k+1} = x_k + \alpha_k s_k$
    $r_{k+1} = r_k - \alpha_k A s_k$
    $\beta_{k+1} = r_{k+1}^T r_{k+1} / r_k^T r_k$
    $s_{k+1} = r_{k+1} + \beta_{k+1} s_k$

Convergence

- A-norm of the error is minimized in each step
- Bound:

$$\|e_k\|_A \leqslant 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|e_0\|_A$$

- κ is the condition number
- for symmetric matrices, $\kappa = \frac{\lambda_{max}}{\lambda_{min}}$

Preconditioning

We've seen iterations that replace A with a matrix Q such that $Q^{-1}A$ resembles the identity matrix. Conjugate Gradient takes a different approach and solves $Ax = b$ more directly.
Can we combine these ideas?
Preconditioning is transforming the original problem $Ax = b$ with another matrix Q and then solving that modified problem. For example

- Left Preconditioning: $QAx = Qb$
- Right Preconditioning: $AQw = b$, $Qw = x$
- Symmetric preconditioning: $Q^TAQw = Q^Tb$, $Qw = x$

For Conjugate Gradient, the matrix must be symmetric, so the last form is the most common.
There are methods that generalize conjugate gradient for nonsymmetric matrices.

Coming Up

- Google, Markov Chains, intro to Monte Carlo Simulations
- Eigenvalues: SVD, Power Method
- Least-Squares

Google (basics)

- A component of Google's success can be attributed to the PageRank^{TM} algorithm developed by Google's founders
- algorithm determined entirely by link structure of the WWW
- recomputed once a month
- involves no content of any Web page
- How are the search results ordered?

Randomly Walking with Google

- start at any webpage
- randomly select a link and follow
- repeat
- what are the outcomes?

The outcomes of such a random walk are:

- a dead end on a page with no outgoing links
- a cycle where you end up where you began: known as a *Markov chain* or *Markov process*.
- The limiting probability that an infinitely dedicated random surfer visits any particular page is its PageRank.
- A page has high rank if other pages with high rank link to it.

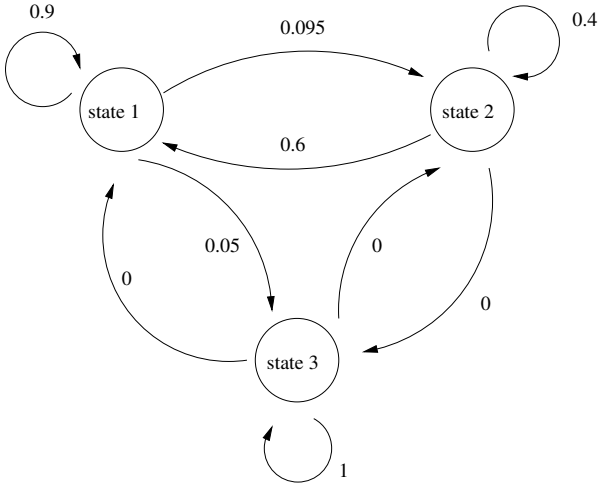
Markov Chains

- Markov chains can model the behavior of a system that depends only on the previous experiment or state.
- That is, the next state of the system depends only on the current state where the outcome of each experiment is one of a discrete set of states.
- Markov chains require a transition matrix, P, where $P(j, i)$ equals the probability of going from state i to state j.
- Each column of P adds up to 1 (the system must end up somewhere).
- If $v^{(0)}$ is the current state distribution, then after n steps

$$v^{(n)} = P^n v^{(0)}.$$

Markov chain model of a Broken Machine

A certain office machine has three states: Working, Temporarily Broken, and Permanently Broken. Each day it is in one of these states. The *transition diagram* between the states is shown below.



Therefore, the transition matrix is:

$$P = \begin{pmatrix} .9 & .6 & 0 \\ .095 & .4 & 0 \\ .005 & 0 & 1 \end{pmatrix}$$

Answers via Markov chains

In our previous model suppose we ask the question:

If the machine is working on Day 1, what is the probability that it is still functional (that is, not permanently broken) a year from now?

Stepping through Time with Markov

We can answer this question by computing the 365'th power of the transition matrix:

$$P^{365} = \begin{pmatrix} .9 & .6 & 0 \\ .095 & .4 & 0 \\ .005 & 0 & 1 \end{pmatrix}^{365} = \begin{pmatrix} 0.1779 & 0.1792 & 0 \\ 0.0284 & 0.0286 & 0 \\ 0.7937 & 0.7922 & 1 \end{pmatrix}$$

The first column of this matrix gives the probability of being in each state on Day 366 (with our assumption that it is working on Day 1, so $v^{(0)} = [1\ 0\ 0]^T$). We see that the probability that the machine is permanently broken is 0.7937, so the chance it is still functional is

$$S(365) = 1 - (v^{(365)})_3 \approx 0.21,$$

i.e., about a 21% chance.

Steady State Solution

For this Markov chain the steady state solution is the vector $\mathbf{v} = [0\ 0\ 1]^T$, as we can see by computing the eigenvalues and eigenvectors. That is, $P\mathbf{v} = \mathbf{v}$.

State 3 (Permanently Broken) is called an *absorbing state*. No matter what state we start in, we will eventually end up in State 3 with probability 1, and once in State 3 we can never leave.

Transient and Absorbing States

- **Transient states:** The machine can leave these. (Here: State 1 = Working, State 2 = Temporarily Broken)
- **Absorbing state:** Once entered, you never leave. (Here: State 3 = Permanently Broken)

Writing the transition matrix with rows as "from" states, $\tilde{P} = P^T$, we can separate it into parts:

$$\tilde{P} = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$$

where

$$Q = \begin{pmatrix} 0.9 & 0.095 \\ 0.6 & 0.4 \end{pmatrix}, \quad R = \begin{pmatrix} 0.005 \\ 0 \end{pmatrix}.$$

Q shows how the machine moves among the "non-broken" states; R gives the probability of moving into the permanently broken state.

The Fundamental Matrix

- The **fundamental matrix** is

$$N = (I - Q)^{-1} = I + Q + Q^2 + Q^3 + \cdots$$

  Each power of Q represents one more "day" of transitions before the machine is permanently broken.
- N_{ij}: expected number of days the machine is in state j if it started in state i (before it breaks forever).
- The sum of row i = average number of days until failure, starting from state i.

For our machine:

$$N = \begin{pmatrix} 200 & 31.67 \\ 200 & 33.33 \end{pmatrix}$$

- Starting from **Working**: about 232 days on average.
- Starting from **Temp. Broken**: about 233 days on average.

Will the Machine Eventually Break?

- The long-term chance of ending up in the **permanently broken** state is

$$B = NR.$$

For our machine:

$$B = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Meaning:

- No matter where the machine starts (working or temporarily broken), it will **eventually end up permanently broken**.
- The only difference is **how long it takes**.

Try It Yourself: MATLAB or Python

MATLAB

```
P = [ .9 .6 0; .095 .4 0; .005 0 1 ];
v0 = [1; 0; 0];
v365 = P^365 * v0;        % After 365 days
survive = 1 - v365(3);    % Still functional
```

Python

```
import numpy as np
P = np.array([[.9,.6,0],[.095,.4,0],[.005,0,1]])
v0 = np.array([1,0,0.0])
v365 = np.linalg.matrix_power(P, 365) @ v0
survive = 1 - v365[2]
```

Both give about a **21% survival chance** after one year.

Back to Google

- Let W be the set of Web pages that can be reached by following a chain of hyperlinks starting from a page at Google.
- Let n be the number of pages in W.
- The set W actually varies with time, by the end of 2020, n was over 40 billion.
- Let G be the $n \times n$ connectivity matrix of W, that is, $G_{i,j}$ is 1 if there is a hyperlink from page j to page i and 0 otherwise.
- The matrix G is huge, but very sparse; its number of nonzeros is the total number of hyperlinks in the pages in W.

Google and Probability

- Let c_j and r_i be the column and row sums of G, respectively. That is,

$$c_j = \sum_i G_{i,j}, \qquad r_i = \sum_j G_{i,j}$$

- Then c_k and r_k are the outdegree and indegree of the k-th page. In other words, c_k is the number of links from page k and r_k is the number of links into page k.
- Let p be the fraction of time that the random walk follows a link.
- Google typically takes this to be $p = 0.85$.
- Then $1 - p$ is the fraction of time that an arbitrary page is chosen.

Google meets Markov

- Let A be an $n \times n$ matrix whose elements are $A_{i,j} = pG_{i,j}/c_j + \delta$ where $\delta = (1-p)/n$.
- This matrix is the transition matrix of the Markov chain of a random walk!
- Notice that A comes from scaling the connectivity matrix by its column sums.
- The j-th column is the probability of jumping from the j-th page to the other pages on the Web.

The problem

Can write A, the transition matrix, as

$$A = pGD + ez^T$$

where e is the vector of all ones and where ez^T accounts for dead-end pages and

$$D_{jj} = 1/c_j \text{ (or 0)} \quad z_j = \delta \text{ (or } 1/n)$$

Then $x = Ax$ can be written (scaling x so that $z^T x = 1$)

$$(I - pGD)x = (z^T x)e = e$$

see `pagerank.m`

Eigenvectors and Google

Find $x = Ax$ and the elements of x are Google's PageRank. Remember $n > 4 \times 10^{10}$ (as of 2020) and growing.

For any particular query, Google finds pages on the Web that match the query. The pages are then listed in the order of their PageRank.

Matlab

Moler's scripts

To search from a homepage, you will type a statement like:

```
[U,G] = surfer('http://usu.edu',n).
```

This starts at the given URL and tries to surf the Web until it has visited n pages. That is, an n by n matrix is formed.

Note, `surfer.m` is very primitive...

Matlab

Moler's scripts

- Download `surfer.m`, `pagerank.m` and `pagerankpow.m`
- `[U,G] = surfer('http://www.slashdot.org',20);`

  U = a cell array of n strings, the URLs of the nodes.

  G = an n-by-n sparse matrix with $G(i,j) = 1$ if node j is linked to node i.
- Next: `spy(G)`. This shows the nonzero structure of the connectivity matrix.
- Finally: `pagerank(U,G)` and the pagerank will be computed.

Numerics?

- what does this have to do with numerical methods?
- large sparse matrices (although Google avoids this)
- solution to a linear system (eigenvalue problem)
- Markov chains...

The Markov process is important. Let's step back and look at some statistics

- randomness
- Monte Carlo
- Markov chains
- Metropolis algorithm

Brownian Motion is an example of a Markov process:
`https://labs.minutelabs.io/Brownian-Motion/` (more later)

Why

- Central to the PageRank (and many many other applications in finance, science, informatics, etc) is that we randomly process something
- what we want to know is “on average” what is likely to happen
- what would happen if we have an infinite number of samples?
- next: eigenvalues, SVD, pagerank, Monte Carlo